

✓ 2. Pandas

Introduction

Pandas is a powerful open-source Python library used for data manipulation and analysis. It provides two primary data structures: **Series** and **DataFrame**, which make it easy to organize and analyze structured data. Pandas allows users to read and write data from various file formats such as CSV, Excel, JSON, SQL databases, and more. It offers powerful functions for cleaning, filtering, sorting, merging, grouping, and transforming datasets. Pandas integrates seamlessly with NumPy, Matplotlib, Seaborn, and Scikit-Learn, making it one of the most important libraries in Data Science and Machine Learning. Due to its efficiency and ease of use, Pandas is widely used for data preprocessing before building Machine Learning models.

Import Library

```
import pandas as pd
```

Common Functions

1. **pd.DataFrame()** – Creates a DataFrame.
2. **pd.Series()** – Creates a Series.
3. **head()** – Displays the first five rows.
4. **tail()** – Displays the last five rows.
5. **info()** – Displays dataset information.
6. **describe()** – Generates statistical summary.
7. **shape** – Returns the number of rows and columns.
8. **columns** – Displays column names.
9. **index** – Displays row indexes.
10. **dtypes** – Shows data types of each column.
11. **isnull()** – Detects missing values.
12. **fillna()** – Replaces missing values.
13. **dropna()** – Removes missing values.
14. **drop()** – Removes rows or columns.
15. **rename()** – Renames columns or indexes.
16. **sort_values()** – Sorts data by column values.
17. **sort_index()** – Sorts data by index.
18. **groupby()** – Groups data based on column values.
19. **value_counts()** – Counts unique values.

- 20. **unique()** – Returns unique values.
- 21. **duplicated()** – Detects duplicate rows.
- 22. **drop_duplicates()** – Removes duplicate rows.
- 23. **sample()** – Returns random rows.
- 24. **concat()** – Combines DataFrames.
- 25. **merge()** – Joins DataFrames using common columns.

```
import pandas as pd

# Sample DataFrame
data = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Alice"],
    "Age": [20, 22, 21, 23, 20],
    "Marks": [85, 90, 88, 92, 85]
}

df = pd.DataFrame(data)

# 1. DataFrame()
print("\n1. DataFrame:\n", df)

# 2. Series()
print("\n2. Series:\n", pd.Series([10,20,30]))

# 3. head()
print("\n3. Head:\n", df.head())

# 4. tail()
print("\n4. Tail:\n", df.tail())

# 5. info()
print("\n5. Info:")
df.info()

# 6. describe()
print("\n6. Describe:\n", df.describe())

# 7. shape
print("\n7. Shape:", df.shape)

# 8. columns
print("\n8. Columns:", df.columns)

# 9. index
print("\n9. Index:", df.index)

# 10. dtypes
print("\n10. Data Types:\n", df.dtypes)

# 11. isnull()
print("\n11. Missing Values:\n", df.isnull())
```

```
# 12. fillna()
df2 = df.copy()
df2.loc[1, "Marks"] = None
print("\n12. Fill Missing Values:\n", df2.fillna(0))

# 13. dropna()
print("\n13. Drop Missing Values:\n", df2.dropna())

# 14. drop()
print("\n14. Drop Column:\n", df.drop(columns=["Marks"]))

# 15. rename()
print("\n15. Rename Column:\n", df.rename(columns={"Marks": "Score"}))

# 16. sort_values()
print("\n16. Sort by Age:\n", df.sort_values(by="Age"))

# 17. sort_index()
print("\n17. Sort Index:\n", df.sort_index(ascending=False))

# 18. groupby()
print("\n18. Group By Name:\n", df.groupby("Name")["Marks"].mean())

# 19. value_counts()
print("\n19. Value Counts:\n", df["Name"].value_counts())

# 20. unique()
print("\n20. Unique Names:", df["Name"].unique())

# 21. duplicated()
print("\n21. Duplicated Rows:\n", df.duplicated())

# 22. drop_duplicates()
print("\n22. Remove Duplicates:\n", df.drop_duplicates())

# 23. sample()
print("\n23. Random Sample:\n", df.sample(2))

# 24. concat()
df_new = pd.DataFrame({"Name": ["Eva"], "Age": [24], "Marks": [95]})
print("\n24. Concatenate:\n", pd.concat([df, df_new], ignore_index=True))

# 25. merge()
student = pd.DataFrame({
    "ID": [1, 2, 3],
    "Name": ["Alice", "Bob", "Charlie"]
})

marks = pd.DataFrame({
    "ID": [1, 2, 3],
    "Marks": [85, 90, 88]
})

print("\n25. Merge:\n", pd.merge(student, marks, on="ID"))
```

```
1. DataFrame:
      Name  Age  Marks
0    Alice   20    85
1     Bob   22    90
2  Charlie   21    88
3    David   23    92
4    Alice   20    85

2. Series:
0    10
1    20
2    30
dtype: int64

3. Head:
      Name  Age  Marks
0    Alice   20    85
1     Bob   22    90
2  Charlie   21    88
3    David   23    92
4    Alice   20    85

4. Tail:
      Name  Age  Marks
0    Alice   20    85
1     Bob   22    90
2  Charlie   21    88
3    David   23    92
4    Alice   20    85

5. Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  -
0   Name    5 non-null        object
1   Age      5 non-null        int64
2   Marks    5 non-null        int64
dtypes: int64(2), object(1)
memory usage: 252.0+ bytes

6. Describe:
      Age      Marks
count  5.000000  5.000000
mean   21.200000  88.000000
std     1.30384   3.082207
min     20.000000  85.000000
25%     20.000000  85.000000
50%     21.000000  88.000000
75%     22.000000  90.000000
max     23.000000  92.000000

7. Shape: (5, 3)

8. Columns: Index(['Name', 'Age', 'Marks'], dtype='object')

9. Index: RangeIndex(start=0, stop=5, step=1)
```

Start coding or [generate](#) with AI.